Coding Assignment 1

Household support: implement, inspect, and verify

Research Methods 2026 | Swapnil Singh

Deadline Monday 9 November 2026, 23:59 (Europe/Vilnius)
Weight 100 assignment marks; contributes 10% of the whole course grade
Work Individual; plan approximately three hours
Hand in `exercise_01.ipynb` and `README.md` through a GitHub pull request

Scope. Use the material from Lectures 1–4: running a project, basic Python, conditions, loops, functions, inline functions (`lambda`), and checks. All inputs are Python lists supplied below. No CSV import, pandas, NumPy, plotting, or external data is required. Git commands and version-control knowledge are not assessed. Use built-in Python tools and, for floating-point checks, `math.isclose`. List comprehensions and `try`/`except` are not required.

AI assistance is permitted.

Inputs and policy specification

These eight households are invented for this assignment. Income and consumption are EUR per month; household size is persons. Positions in the four lists refer to the same household. Preserve the supplied lists.

```
household_ids = ["A01", "A02", "A03", "A04",
                 "A05", "A06", "A07", "A08"]
income_text = ["1200", "0", "1500", "700",
               "2400", "800", "0", "1800"]
consumptions = [1250, 150, 1600, 850, 2600, 950, 0, 1400]
household_sizes = [2, 1, 3, 1, 4, 2, 2, 3]
```

For income I, consumption C, and size n, the hypothetical monthly transfer is

$$T(I, C, n) = \begin{cases} 0, & I/n > 600, \\ \min\{100n,\ \max\{C - I, 0\}\}, & I/n \le 600. \end{cases}$$

Thus eligibility includes exactly EUR 600 per person. An eligible household receives its consumption shortfall, capped at EUR 100 per person. A household without a shortfall receives zero. Consumption is held fixed throughout.

Define income after support as $Y = I + T$, saving before as $S = I - C$, and saving after as $S' = Y - C$. A saving share is saving divided by the corresponding income; it is `None` when that income is zero. Negative saving is valid.

1. Trace values and notebook state [12 marks]

The following snippet is separate from the eight-household inputs. Before running it, write your predictions in a Markdown cell. Keep that first attempt, then run the snippet and explain any corrections. Initial prediction errors do not themselves lose marks if the final reasoning is correct.

```
incomes = [1000, 800, 0]
saved_incomes = incomes
first_share = (incomes[0] - 800) / incomes[0]
incomes[0] = 1200
saved_incomes.append(600)
print(incomes)
print(saved_incomes[-1], len(incomes))
print(f"{first_share:.0%}")
```

(a) Give the three output lines and trace the relevant values. [4]

(b) Explain why appending through `saved_incomes` affects `incomes`, and why changing an income does not update `first_share`. [4]

(c) Make `saved_incomes` an independent copy of the original list and recalculate the share after changing income to 1200. Show your changes and the new three output lines. [4]

2. Implement the policy as functions [28 marks]

Write the following three functions. Return values rather than only printing them. Assume amounts are finite real numbers and household sizes are integers; you do not need general type, NaN, or infinity checks.

(a) `saving_share(income, consumption)` [8]
Raise `ValueError` if either amount is negative. Otherwise return `None` for zero income, and $(I - C)/I$ for positive income. Check invalid amounts *before* the zero-income case. Return a fractional share, not a percentage.

(b) `transfer(income, consumption, size)` [12]
Raise `ValueError` for negative income, negative consumption, or `size <= 0`, before calculating eligibility. For valid inputs, implement the policy on page 1 exactly. Every valid call must return a number, including zero when no transfer is payable.

(c) `post_policy_share(income, consumption, size)` [8]
Call your `transfer` function, add its result to original income, and call `saving_share` using that new income and unchanged consumption. Return the result. Reuse both helpers; do not duplicate their policy or share formulas.

Explain your design in at most 100 words: distinguish parameters from arguments, explain why `return` is needed, and explain why input checks precede division. Functions must use their arguments rather than depend on remembered notebook values. Place these explanations beside the relevant code. The marks above cover both implementation and explanation.

3. Build and interpret a checked report [30 marks]

Use your functions from Question 2. Write explicit loops where requested so that your reasoning is visible. Keep full precision in calculations and round only for display. Counts must identify their denominator or population.

(a) **Prepare inputs.** Convert `income_text` to a new list of numbers using a loop; leave the original strings unchanged. Check that all four input lists have equal lengths *before* processing. Raise `ValueError` for a length mismatch; do not let `zip` silently omit households. [4]

(b) **Calculate household results.** Use an explicit loop over the aligned inputs. Build result lists and print a labelled report with one line per household containing: ID, original income, consumption, size, transfer, saving before, saving after, share before, and share after. Use two decimal places for EUR amounts and percentage displays; print `undefined` for `None`. Do not replace an undefined share with zero. [8]

(c) **Compare totals and averages.** Calculate and report: [8]

- total transfer, number of households receiving a positive transfer, and numbers with negative saving before and after;
- the pooled saving share before, $\sum_i S_i / \sum_i I_i$, and after, $\sum_i S_i' / \sum_i Y_i$;
- the arithmetic mean of defined household saving shares before and after, with the number included in each mean;
- the mean after support restricted to households whose share was already defined *before* support, using the same households for both periods.

In at most 120 words, compare the pooled shares and household means. Identify any change in which households enter the means and explain why comparing means over different sets can be misleading. No statistical method beyond totals, counts, and arithmetic averages is needed.

(d) **Use an inline function.** Build a new list containing only the defined *pre-policy* household shares. Use `sorted` with a `lambda` key to order them by distance from a target saving share of `0.10`, nearest first. Print this list and explain what the lambda receives and returns. Retain the original share values, including their signs; do not output the distances instead. Preserve the original result list, including `None`. [4]

(e) **Test a counterfactual.** Make an independent copy of the numeric income list and change only A01's income from 1200 to 1202. Recompute A01's transfer, income after support, and saving after support. Show these quantities under both inputs and the change in each. Explain in at most 80 words whether higher original income necessarily gives higher income after support under this policy. Verify that your original input list remains unchanged. [6]

Interpretation matters. Distinguish a change in a household's saving amount from a change in its saving share. A program that runs or prints a plausible number does not establish that it answers the stated question.

4. Audit an agent-written function [20 marks]

An agent proposes the function below. Its intended contract is to return the arithmetic mean of *defined* saving shares, including valid zero and negative shares. It must raise `ValueError` for unequal list lengths and return `None` when no share is defined, including empty inputs. It must not modify its inputs. Assume each individual input pair meets Question 2's numeric and non-negative conditions. `saving_share` is your correct helper.

```python
def proposed_mean(incomes, consumptions):
    total = 0
    count = 0
    for income, consumption in zip(incomes, consumptions):
        share = saving_share(income, consumption)
        if share is not None and share != 0:
            total = 0
            total = total + share
            count = count + 1
    return total / count
```

(a) Identify **four distinct defects** and connect each to the intended contract. Do not count the same defect twice. [8]

(b) Write a repaired `mean_defined_share(incomes, consumptions)` using a loop and your helper. Preserve the intended contract. [6]

(c) Give four small diagnostic input pairs. For each, state the expected correct outcome, what the proposal actually does, and which defect it exposes. Use cases that separate the defects as far as possible. An outcome may be a number, `None`, or an exception type. [6]

Put deliberately failing calls and their observed errors in Markdown code blocks in the final notebook. You may investigate them in scratch cells first; you do not need to learn `try/except` to finish this task.

5. Supply evidence and a reproducible hand-in [10 marks]

(a) Include at least **eight meaningful, passing assertions** covering ordinary saving, negative saving, zero income, the eligibility boundary, the transfer cap, no shortfall, all-undefined shares, and the counterfactual's preserved original input. Use `is None` for undefined results and `math.isclose` for floating-point comparisons. For four assertions, explain in one sentence which plausible mistake each would detect. [6]

(b) Restart the kernel and run all cells in order. The final notebook must run without manual intervention or unhandled exceptions, and reproduce all reported results. Keep the original first prediction from Question 1 in Markdown. [2]

(c) In `README.md`, give the Python version, instructions to open and run the notebook, and a brief AI-use record (or a no-AI statement). Submit the notebook and README through a pull request. [2]